AutoDSC: An Automated Data Stream Clustering Framework

Anonymous

***Abstract*—Data stream clustering faces unique challenges in dynamic environments where data distributions evolve continuously and computational resources are limited. Traditional clustering algorithms are sensitive to algorithm selection, hyperparameter configuration, and changes in data characteristics, yet while AutoML has transformed supervised learning, its application to unsupervised streaming tasks remains largely unexplored and the few existing solutions rely on single-metric evaluation or computationally intensive offline search. We introduce AutoDSC (Automated Data Stream Clustering), an AutoML framework that casts online stream clustering as a combined algorithm selection and hyperparameter optimization (CASH) problem and solves it in a single pass as the data distribution drifts. To propose configurations, AutoDSC uses Population-Guided Stochastic Search Optimization (PGSSO), which guides exploration from a recency-weighted elite archive and adjusts its exploration rate automatically from performance feedback, recovering implicitly after drift. To choose among configurations without labels, it scores each one with a normalized multi-objective criterion that combines three complementary internal cluster-validity indices. Across ten real-world and synthetic streams, AutoDSC attains the best mean rank on both the Adjusted Rand Index and CMM and the highest mean ARI among all methods, significantly outperforming untuned single algorithms, while remaining the most computationally efficient of the automated approaches, processing 2–3$\times$ more instances per second than competing AutoML stream clusterers at a lower memory footprint.**

***Index Terms*—data stream clustering, AutoML, concept drift, hyperparameter optimization, online learning**

I. Introduction

Automated Machine Learning (AutoML) has significantly advanced the accuracy and efficiency of machine learning workflows by automating tasks such as algorithm selection, hyperparameter optimization, and pipeline synthesis [1]. While AutoML has seen widespread adoption in supervised learning, its application to clustering remains comparatively less explored, mainly due to the absence of a universally applicable objective function for evaluation [2]. Recent AutoML solutions for clustering include AutoML4Clust [3], AutoCluster [4], cSmartML [5], ML2DAC [6], TPE-AutoClust [7], and PoAC [8]. These approaches address clustering challenges using various automated methodologies such as meta-learning, pipeline composition, and problem-specific heuristics.

However, with the increasing availability of streaming data from sources like Internet of Things (IoT) sensors, web traffic, and social media, traditional static AutoML approaches are insufficient [9]. They cannot adapt to dynamic environments or process data in a single pass due to concept drift and memory limitations [10]. Existing online AutoML frameworks such as Online AutoML [11], ASML [12], ChaCha [13], and AutoReg [14] primarily focus on supervised learning tasks. Meanwhile, unsupervised learning, particularly clustering, requires specialized AutoML solutions. At present, ConfStream [15] and OnlineAutoClust [16] represent the only established AutoML (CASH-style) solutions for stream clustering; the recent MOStream [17] also self-optimizes, but from a rule-based angle that reconfigures structural components rather than searching the joint algorithm–hyperparameter space. Nevertheless, the AutoML solutions suffer from significant limitations. ConfStream relies on single clustering evaluation metrics, and OnlineAutoClust uses computationally intensive offline search methods, which are inefficient for real-time streaming scenarios. These limitations highlight the critical need for tailored AutoML solutions designed specifically for data stream clustering [18]. This paper introduces AutoDSC (Automated Data Stream Clustering), an AutoML framework that brings automated algorithm selection and hyperparameter optimization to unsupervised data streams and adapts continuously as the data distribution evolves. We make the following key contributions.

- We cast online stream clustering as a streaming combined algorithm selection and hyperparameter optimization (CASH) problem that jointly chooses the clustering algorithm and tunes its hyperparameters while the data distribution drifts, and we present AutoDSC, a single-pass AutoML framework that solves it autonomously without manual tuning or labels.
- To propose new configurations, AutoDSC introduces Population-Guided Stochastic Search Optimization (PGSSO), which maintains a recency-weighted elite archive and an adaptively controlled exploration rate so that promising algorithm and hyperparameter settings are found quickly and the population recovers implicitly as the stream drifts.
- To choose among configurations without ground-truth labels, AutoDSC scores each one with a normalized multi-objective criterion that combines three complementary internal cluster-validity indices, yielding robust selection where the single-metric criteria of prior systems fail.
- Across ten real and synthetic streams, we show that AutoDSC attains the best mean rank on both the Adjusted Rand Index (ARI) [19] and CMM [20] and the highest mean ARI among all methods while remaining the most computationally efficient automated approach.

II. Related Work

Data stream clustering addresses the challenges of processing continuous, high-velocity data with limited computational resources [10], [21]. CluStream [22] pioneered the micro-cluster approach using pyramidal time frames to maintain summary statistics for scalable analysis. DenStream [23] advanced density-based clustering by introducing core-micro-clusters, potential micro-clusters, and outlier micro-clusters. DBSTREAM [24] enhanced density estimation by incorporating shared density graphs that explicitly model inter-micro-cluster relationships. This provides better resilience to concept drift compared to DenStream. For k-means-based approaches [25], mini-batch optimization combined with sparse projections has been developed to enable web-scale clustering with significantly reduced latency for real-time streaming applications. StreamKMeans [26] is an alternative version of the stream local search [27] that replaces k-medians with incremental k-means.

In the batch setting, several AutoML systems automate clustering, such as AutoML4Clust [3], AutoCluster [4], cSmartML [5], ML2DAC [6], TPE-AutoClust [7], and PoAC [8], using meta-learning and pipeline composition. However, all of them assume repeated full-data access and therefore cannot operate under the single-pass, bounded-memory, drifting conditions of a data stream. Recent developments have focused on automated algorithm selection and hyperparameter optimization (CASH) [1] for stream clustering. ConfStream [15] represents an early attempt at automated stream clustering configuration, using an ensemble of different clustering algorithms and configurations that dynamically adapt by evaluating clustering quality and sampling nearby new configurations proportionally to their performance using a normal distribution. However, ConfStream has significant limitations, including its reliance on a single Silhouette Width metric for quality assessment and limited parameter search capabilities. OnlineAutoClust [16] developed an AutoML framework that combines Bayesian optimization with clustering ensembles, employing Tree-structured Parzen Estimators (TPE) for algorithm selection and hyperparameter tuning. However, OnlineAutoClust suffers from critical limitations, including dependence on offline tuning methods, high computational requirements, and sensitivity to allocated time budgets, which can lead to suboptimal pipeline selections during rapid concept drift. From a different, rule-based angle, MOStream [17] pursues self-optimization by decomposing a stream clustering algorithm into four design dimensions, namely the summarizing data structure, the window model, the outlier-detection mechanism, and the cluster-refinement strategy, and adaptively reconfiguring these components at runtime to balance clustering accuracy against processing throughput. Unlike the CASH-style frameworks above, MOStream tunes structural design choices rather than searching a joint algorithm–hyperparameter space. In contrast to all of these approaches, AutoDSC adapts to drift implicitly through continual per-window re-evaluation and an adaptive exploration rate, avoiding both single-metric evaluation and expensive offline optimization.

Because ground-truth labels are unavailable during streaming, configuration selection must instead rely on internal cluster validity indices (CVIs) such as the Silhouette, Davies–Bouldin, and Calinski–Harabasz scores; since no single CVI is reliable across varying cluster shapes, densities, and counts, combining complementary indices is more robust than depending on any one alone [28], [29], which is precisely the weakness of ConfStream's single-metric criterion. External evaluation, by contrast, requires measures that account for the evolving nature of the data, since classical external indices assume a static partition and cannot correctly attribute errors arising from emerging, splitting, or moving clusters [20]. To address this, Kremer et al. [20] introduced the Cluster Mapping Measure (CMM), a ground-truth measure that combines penalties for missed objects, misplaced objects, and noise inclusion, weighted by object connectivity, to robustly reflect the error types specific to evolving streams. We therefore adopt CMM, alongside the chance-corrected Adjusted Rand Index, to assess clustering quality (Section V).

III. Automated Data Stream Clustering

We introduce Automated Data Stream Clustering (AutoDSC), a novel AutoML framework tailored for non-stationary data stream clustering. AutoDSC processes data in fixed-size windows. At the end of each window, AutoDSC evaluates clustering configurations using multiple normalized validation indices. The framework maintains a recency-weighted elite archive of high-performing configurations to guide future parameter exploration via PGSSO. It adaptively balances exploration and exploitation through dynamic rate adjustment based on performance feedback. Adaptation to distributional change is implicit, since each window the population is re-evaluated and the incumbent re-selected, and a drop in clustering quality automatically raises the exploration rate so new configurations are sought for the new regime. Figure 1 illustrates the high-level workflow of AutoDSC, while Algorithm 1 details the complete operational steps.

AutoDSC processes an infinite data stream $\mathcal{S} = \{x_1, x_2, \ldots, x_t, \ldots\}$ where each instance $x_t \in \mathbb{R}^d$ arrives at timestamp t through a fixed-size windowing mechanism $W_t = \{x_{t-w+1}, x_{t-w+2}, \ldots, x_t\}$ with capacity w. The framework maintains a configuration population $\mathcal{A}_t$ containing multiple clustering algorithms with different hyperparameter settings, continuously adapts these configurations based on performance feedback, and provides real-time predictions using the best-performing configuration. The configuration space $\Theta = \prod_{i=1}^{|\mathcal{A}|} \Theta_i$ encompasses parameter spaces across all algorithms in the portfolio $\mathcal{A}$, where each configuration $\theta = (a_i, \phi_i)$ specifies both algorithm selection $a_i \in \mathcal{A}$ and hyperparameter values $\phi_i \in \Theta_i$. As each new data point arrives, the system performs incremental updates across all configurations:

$$\forall \theta_i \in \mathcal{A}_t : \quad M_i^{(t)} = \text{LearnOne}(M_i^{(t-1)}, x_t) \tag{1}$$

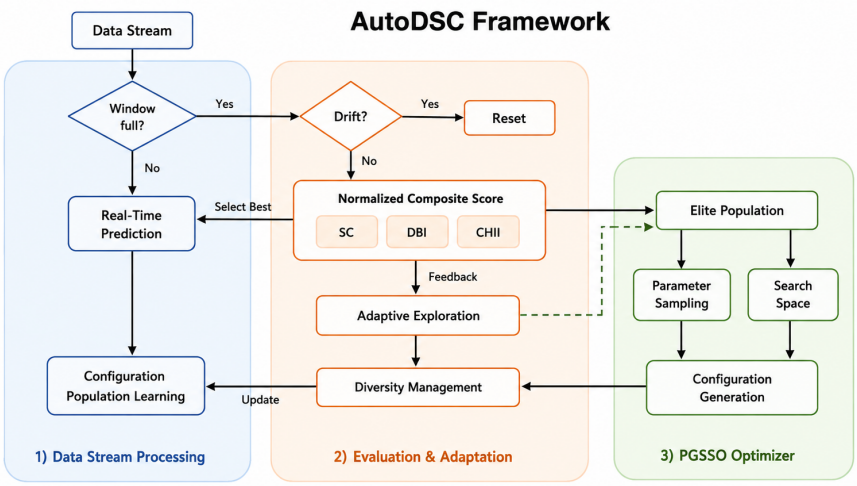


Fig. 1. AutoDSC Framework Overview

where $M_i^{(t)}$ represents the model state of configuration θ_i at time t. When the window reaches capacity, the framework initiates evaluation and optimization operations.

When the window fills, AutoDSC evaluates clustering quality using three normalized clustering validation indices, namely the Normalized Silhouette Coefficient $\text{NSC} = (1 + \text{SC})/2$, the Normalized Davies-Bouldin Index $\text{NDBI} = 1 - \tanh(\text{DBI})$, and the Normalized Calinski-Harabasz Index $\text{NCHI} = \tanh(\log(1 + \text{CHI})/10)$, ensuring all metrics are bounded to $[0, 1]$ where higher values indicate better clustering quality. Each configuration is scored on the entire window (the per-point cluster assignments produced during LEARNONE are retained for this purpose), so no sub-sampling is required. The expected clustering quality for configuration θ at time t is:

$$\mu_t(\theta) = \frac{1}{3}[\text{NSC} + \text{NDBI} + \text{NCHI}] \tag{2}$$

AutoDSC dynamically adjusts exploration behavior based on performance improvement trends. The exploration rate α_t adapts based on relative improvement:

$$\Delta\mu_t = \mu_t(\theta_t^*) - \mu_{t-1}(\theta_{t-1}^*) \tag{3}$$

$$\alpha_{t+1} = \begin{cases} \max(0.1,\ \alpha_t \cdot (1 - \beta_t)) & \text{if } \Delta\mu_t > 0 \\ \min(1.0,\ \alpha_t \cdot (1 + \beta_t)) & \text{if } \Delta\mu_t \leq 0 \end{cases} \qquad \beta_t = \min(\gamma, |r_t|) \tag{4}$$

where $r_t = \Delta\mu_t / |\mu_{t-1}|$ is the relative improvement and β_t is the resulting step, capped at $\gamma = 0.5$ for stability. This parameter-free mechanism decreases exploration when performance improves (favoring exploitation) and increases it when performance stagnates or drops, as happens under concept drift, encouraging broader search to recover.

The framework employs **Population-Guided Stochastic Search Optimization (PGSSO)** (Algorithm 2) to jointly optimize algorithm selection and hyperparameter tuning. PGSSO maintains an elite archive $\mathcal{E}_t = \{(\theta_i, s_i)\}_{i=1}^{e}$ where each configuration $\theta_i = (a_i, \phi_i)$ pairs an algorithm choice with its hyperparameters and $s_i = \mu_t(\theta_i)$ is its score. To remain responsive to drift, elites are ranked by a *recency-weighted* score $s_i/(1 + g - g_i)$, where g is the current generation and g_i the generation at which θ_i was last scored; stale high-scoring configurations are thus down-weighted in favor of recently effective ones. By tracking which algorithm-hyperparameter combinations perform best, PGSSO naturally learns both which algorithms suit current data characteristics and their optimal settings. The elite population encodes this joint preference, where frequently appearing algorithms indicate superior algorithmic choices, while their parameter distributions reveal effective settings. PGSSO adaptively balances exploration and exploitation through dynamic rate α_t, computing statistical distributions over both algorithm selections (via frequencies) and their parameters (via means μ_j and variances σ_j^2), which together guide sampling of new promising algorithm-hyperparameter configurations.

PGSSO generates new configurations through probabilistic sampling:

$$\theta_{\text{new}} = \begin{cases} \text{Elite-Guided}(\mathcal{E}_t) & \text{with probability } (1 - \alpha_t) \\ \text{Random}(\Theta) & \text{with probability } \alpha_t \end{cases} \tag{5}$$

Elite-guided sampling first selects an algorithm based on elite frequencies, then samples its hyperparameters from learned distributions. Algorithm selection uses:

$$P(a = a_k) = \frac{\sum_{(\theta,s)\in\mathcal{E}_t} \mathbf{I}[a = a_k]}{|\mathcal{E}_t|} \tag{6}$$

where $a_k \in \mathcal{A}$ represents available algorithms. Given selected algorithm a_k, numerical hyperparameters use Gaussian distributions from elite configurations of that algorithm:

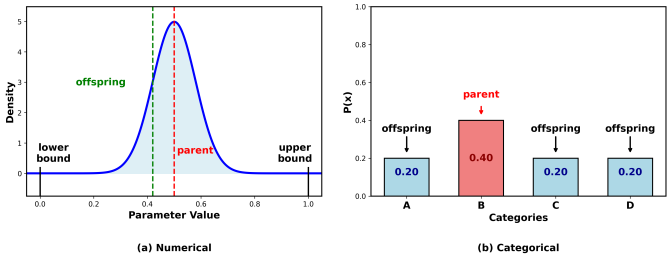


Fig. 2. Parameter Sampling Visualization.

$$\phi_j \sim \mathcal{N}(\mu_j, \sigma_j^2) \text{ where } \mu_j = \frac{1}{|\mathcal{E}_t^{a_k}|} \sum_{(\theta,s)\in\mathcal{E}_t^{a_k}} \phi_j \tag{7}$$

and $\mathcal{E}_t^{a_k} = \{(\theta, s) \in \mathcal{E}_t : a = a_k\}$ contains elite configurations using algorithm a_k. Categorical hyperparameters similarly use frequency-based sampling within algorithm-specific elites. Figure 2 illustrates this hierarchical sampling process.

Population diversity is maintained by computing configuration distances:

$$d(\theta_1, \theta_2) = \begin{cases} 1.0 & \text{if algorithms differ} \\ \frac{1}{|\mathcal{P}|} \sum_{p\in\mathcal{P}} d_p(\theta_1[p], \theta_2[p]) & \text{otherwise} \end{cases} \tag{8}$$

where $\mathcal{P}$ represents common parameters and d_p computes normalized parameter-specific distances. When the population exceeds size K, it is pruned by first retaining the incumbent and top-scoring configurations and then greedily adding the remaining configuration that is *farthest* (by d) from those already kept, a farthest-first selection that preserves diversity without requiring a fixed acceptance threshold.

Throughout the process, AutoDSC provides real-time clustering predictions using the incumbent (best) configuration:

$$\theta_t^* = \arg\max_{\theta_i\in\mathcal{A}_t} \mu_t(\theta_i) \tag{9}$$

A. Computational Complexity

The framework's computational complexity scales efficiently with population size and window parameters. Per-instance processing requires $O(K \cdot C)$ operations where $K = |\mathcal{A}_t|$ and C denotes average algorithm complexity. Window-end evaluation involves $O(K \cdot w \cdot d)$ operations to score each configuration on the full window. Total per-window complexity is $O(K \cdot w \cdot d + K^2)$, where K^2 accounts for diversity-based population management. Crucially, K is fixed, so per-point cost does not grow with stream length. Space complexity is $O((K + e_{\max}) \cdot |\Theta| + w \cdot d)$.

IV. Theoretical Analysis

We analyze AutoDSC using a phase-based approach where phases correspond to periods with fixed incumbent configurations. Let $j \in \{1, ..., T/w\}$ index windows where T/w represents the total number of windows given T instances and window size w, and $t \in \{1, ..., T\}$ index instances. We analyze AutoDSC's performance over T/w windows divided into $M + 1$ phases, where phase m spans N_m windows from window j_m to $j_{m+1} - 1$ with population $\mathcal{A}_{j_m}$ and incumbent configuration $\theta^*_{j_m}$. We track cumulative evaluations $n_{\theta,j}$ for each configuration θ across all windows up to window j. The expected clustering quality for configuration θ at window j is defined as:

Algorithm 1: AutoDSC

Input: Stream $\mathcal{S} = \{x_1, x_2, \ldots\}$, $x_t \in \mathbb{R}^d$; window size w; population size K
Output: Cluster predictions $\{y_t\}_{t=1}^{\infty}$ using θ_t^*

$e_{\max} \leftarrow \lfloor K/2 \rfloor$;
$\mathcal{E}_0 \leftarrow \emptyset$, $\alpha_0 \leftarrow 0.5$;
$\mathcal{A}_0 \leftarrow \{\theta_i\}_{i=1}^K$ where $\theta_i \sim \text{Uniform}(\Theta)$;
$W_0 \leftarrow \emptyset$, $\theta_0^* \leftarrow \emptyset$;
$\mu_{-1}(\theta_{-1}^*) \leftarrow -\infty$;
foreach $x_t \in \mathcal{S}$ **do**
  $W_t \leftarrow W_{t-1} \cup \{x_t\}$;
  foreach $\theta_i \in \mathcal{A}_t$ **do**
    $\ell_{i,t} \leftarrow \text{PredictOne}(M_i, x_t)$; // store assignment
    $M_i^{(t)} \leftarrow \text{LearnOne}(M_i^{(t-1)}, x_t)$;
  if $|W_t| = w$ **then**
    foreach $\theta_i \in \mathcal{A}_t$ **do**
      $L_i \leftarrow \{\ell_{i,\tau} : x_\tau \in W_t\}$;
      $\mu_t(\theta_i) \leftarrow \text{NormCVI}(W_t, L_i)$;
    $\mu_{\text{best}}^{(t)} \leftarrow \max_i \mu_t(\theta_i)$;
    $\alpha_{t+1} \leftarrow \text{Adapt}(\alpha_t, \mu_{t-1}(\theta_{t-1}^*), \mu_{\text{best}}^{(t)})$;
    $\mathcal{C}_{\text{new}} \leftarrow \text{PGSSO}(\{(\theta_i, \mu_t(\theta_i))\}, \alpha_{t+1}, \Theta)$;
    $\mathcal{A}_{t+1} \leftarrow \text{SelectDiverse}(\mathcal{A}_t \cup \mathcal{C}_{\text{new}}, K)$;
    $\theta_t^* \leftarrow \arg\max_{\theta_i\in\mathcal{A}_{t+1}} \mu_t(\theta_i)$;
    $W_t \leftarrow \emptyset$;

Algorithm 2: PGSSO

Input: Current configurations with scores $\{(\theta_i, s_i)\}$; exploration rate $\alpha \in [0, 1]$; parameter space Θ
Output: New configuration set $\mathcal{C}_{\text{new}}$

$\mathcal{E} \leftarrow \text{UpdateElitePopulation}(\{(\theta_i, s_i)\}, e_{\max})$;
$\mathcal{E} \leftarrow \text{UpdateParameterDistributions}(\mathcal{E})$;
$\mathcal{C}_{\text{new}} \leftarrow \emptyset$;
for $j \leftarrow 1$ **to** n_{gen} **do**
  $u \leftarrow \text{Uniform}(0, 1)$;
  if $u > \alpha$ **then**
    $\theta_{\text{new}} \leftarrow \text{SampleFromElite}(\mathcal{E}, \Theta)$;
  else
    $\theta_{\text{new}} \leftarrow \text{RandomSample}(\Theta)$;
  $\mathcal{C}_{\text{new}} \leftarrow \mathcal{C}_{\text{new}} \cup \{\theta_{\text{new}}\}$;
return $\mathcal{C}_{\text{new}}$;

$$\mu_j(\theta) = \mathbb{E}[r_j(\theta)] \tag{10}$$

where $r_j(\theta) \in [0,1]$ represents the instantaneous composite CVI score, and $\theta_j^{\text{opt}} = \arg\max_{\theta \in \Theta} \mu_j(\theta)$ denotes the optimal configuration at window j.

Definition 1: Let $p \in (0,1)$ denote the sample complexity parameter where estimation error decreases as $O(n^{-p})$ with n samples (typically $p = 1/2$ for clustering).

Definition 2: Let comp_θ denote the computational complexity of evaluating configuration θ on one window.

Assumption 1: Rewards are bounded: $r_j(\theta) \in [0,1]$.

Assumption 2: Within stationary segments, $\mu_j(\theta)$ is L-Lipschitz in θ.

Assumption 3: The optimal configuration θ_j^{opt} enters the population within $O(\log|\Theta|)$ windows after each concept drift.

Define confidence intervals:

$$\epsilon_{\theta,j} = O\left(\sqrt{\frac{\log(|\Theta|j/\delta)}{n_{\theta,j}}}\right) \tag{11}$$

where $n_{\theta,j}$ is the number of evaluations of θ by window j and $\delta \in (0,1)$ is a confidence parameter [30].

Lemma 1 (PGSSO Selection Properties): With probability $\geq 1-\delta$ [31]:

Claim 1: If $\mu_j(\theta) - \mu_j(\theta^*_{j_m}) + 2\epsilon_{\theta,j} + 3\epsilon_{\theta^*_{j_m},j} > 0$, then PGSSO selects θ as the new incumbent $\theta^*_{j_{m+1}}$.

Claim 2: When the incumbent switches from $\theta^*_{j_m}$ to $\theta^*_{j_{m+1}}$, we have $\mu_{j_{m+1}}(\theta^*_{j_m}) - \mu_{j_{m+1}}(\theta^*_{j_{m+1}}) > \epsilon_{\theta^*_{j_m},j_{m+1}}$.

Claim 3: If $\mu_j(\theta) < \mu_j(\theta^*_{j_m})$, then θ will not replace $\theta^*_{j_m}$ as the incumbent.

Lemma 2 (Drift Recovery via Adaptive Exploration): When a concept drift degrades incumbent quality ($\Delta\mu_j \leq 0$), the exploration rate increases (Eq. 4). Under Assumption 3, the post-drift optimal configuration re-enters the population within $O(\log|\Theta|)$ windows with probability $\geq 1-\delta$, driven solely by the adaptive exploration rate.

Lemma 3 (Adaptive Exploration): The adaptive exploration mechanism in PGSSO, which adjusts α_j based on performance trends, improves the expected regret constants by a factor of $(1+\gamma)^{-1}$ compared to fixed exploration, where γ is the bounded adaptation gain (the per-step exploration adjustment is capped at $\gamma = 0.5$).

Proposition 1 (Window Regret): With sample complexity parameter p and properly set confidence intervals, AutoDSC achieves:

$$\sum_{m=0}^{M} \sum_{j=j_m}^{j_{m+1}-1} \left(\mu_j(\theta_j^{\text{opt}}) - \mu_j(\theta^*_{j_m})\right) = \tilde{O}\left(\frac{K \cdot \text{comp}_{\theta^{\text{opt}}} \cdot (T/w)^p}{p(1+\gamma)} + \text{comp}_{\theta^*}(T/w)^{\frac{1}{2-p}}\right) \tag{12}$$

where $K = \max_j |\mathcal{A}_j|$ is the maximum population size.

Proof Sketch: The bound decomposes regret into two terms: (i) regret from suboptimal population sampling, bounded by PGSSO's exploration guarantee with adaptive exploration reducing the constant by $(1+\gamma)^{-1}$, and (ii) regret from incumbent updates. Lemma 1 ensures incumbent switches occur only when sufficiently beneficial, limiting the number of phases $M = O((T/w)^{\frac{1-p}{2-p}})$. The confidence intervals shrink as $O(n_{\theta,j}^{-1/2})$, yielding the stated bounds [30].

Theorem 1 (Total Regret): Under the same conditions, AutoDSC achieves instance-level regret:

$$\sum_{t=1}^{T} \left(\mu_{\lceil t/w \rceil}(\theta^{\text{opt}}_{\lceil t/w \rceil}) - \mu_{\lceil t/w \rceil}(\theta^*_{\lceil t/w \rceil})\right) = \tilde{O}\left(\frac{K \cdot \text{comp}_{\theta^{\text{opt}}} \cdot T^p}{(1+\gamma)} + \text{comp}_{\theta^*} T^{\frac{1}{2-p}}\right) \tag{13}$$

Proof Sketch: Since clustering quality is constant within windows, summing over T instances with window-level quality yields:

$$\sum_{t=1}^{T}(\cdot) = \sum_{j=1}^{T/w} w \cdot (\mu_j(\theta_j^{\text{opt}}) - \mu_j(\theta_j^*)) \tag{14}$$

Applying Proposition 1 and noting w is constant gives the result.

Corollary 1 (Drift Recovery): After concept drift, AutoDSC recovers ϵ-optimal performance within $O(\log|\Theta|/\epsilon^2)$ windows.

For $p = 1/2$, the regret bound becomes $\tilde{O}(\sqrt{T})$, matching optimal rates for non-stationary bandits with concept drift [32].

TABLE I
SUMMARY OF DATASETS

Dataset Name	Type	Instances	Attributes	Clusters
Insects	Real	52,848	33	6
KDD99	Real	100,655	38	12
PenDigits	Real	10,992	16	10
Shuttle	Real	58,000	9	7
Vehicle	Real	98,528	100	3
Expanding	Synth	100,000	5	5
Gradual	Synth	100,000	5	5
Merge-Split	Synth	100,000	5	5
Sudden	Synth	100,000	5	5
Fast-RBF	Synth	100,000	5	5

V. EXPERIMENTAL SETUP

Our experimental design follows rigorous evaluation protocols to ensure reproducible and statistically valid results.[1]

Datasets: We selected 10 datasets comprising 5 real-world and 5 synthetic streams with varying sizes, dimensionalities, and cluster counts, as summarized in Table I.

Evaluation Protocol: All experiments use prequential evaluation (test-then-train) [10] with 10 independent runs using random seeds 1-10 to account for stochastic variability. Clustering quality is measured using the Adjusted Rand Index (ARI) and CMM [20], where ground truth labels are used solely for evaluation, not training or model selection. ARI is the chance-corrected form [19] of the Rand index [33] that

[1]Datasets and code: https://anonymous.4open.science/r/AutoDSC

counts agreements between the predicted clustering and the ground-truth partition over all pairs of points and adjusts the count for the agreement expected at random, yielding 1 for a perfect match, 0 for random labelling, and negative values for worse-than-random partitions. We also record throughput (instances/s) and peak memory usage (MB) to assess computational efficiency. Statistical significance is assessed using the Friedman test followed by the Nemenyi post-hoc test [34] with $\alpha = 0.05$.

Baseline Methods: We compare AutoDSC against five baselines spanning three categories. (i) *AutoML stream-clustering methods*: *ConfStream* [15] and *OnlineAutoClust* (OACE) [16], established frameworks that optimize hyperparameters online. (ii) The *rule-based adaptive* method *MOStream* [17]. (iii) Two *baselines*: *RandomSearch* [35], which shares AutoDSC's framework structure but generates configurations by purely random sampling, a direct comparison to our guided optimization, and the *Default(Best)* baseline of OnlineAutoClust [16], which captures that "one of the most commonly-used approaches by non-expert users is to try all possible clustering algorithms with their default hyperparameters and then select the model with the best performance." Following this, we run the four River fixed algorithms (CluStream, DenStream, DBSTREAM, StreamKMeans) with their default settings and report the best-performing model per dataset.

Configuration: Unless otherwise stated, AutoDSC uses its default configuration of population size $K = 10$ and window size $w = 1000$, with the exploration rate adapting automatically from an initial value of 0.5 (Section VI-A analyzes sensitivity to K and w). The AutoML baselines search over the same algorithm portfolio and hyperparameter ranges specified in Table II, where the window-dependent ranges (`time_window` and `chunk_size`) are listed for the default window size $w = 1000$; the single fixed algorithms use their library defaults. Other baseline-specific parameters remain as reported in their original papers.

All experiments were conducted on AMD EPYC 7702 64-core servers with 1TB RAM, utilizing parallelization for computational efficiency. All clustering algorithms were implemented using the River Python library [26] for online machine learning.

TABLE II
SEARCH SPACE CONFIGURATION.

Algorithm	Hyperparameter	Type	Range/Options
CluStream	n_macro_clusters	integer	[2, 15]
	max_micro_clusters	integer	[20, 100]
	time_window	integer	[250, 1500]
	micro_cluster_r_factor	numeric	[1.0, 3.0]
DenStream	decaying_factor	numeric	[0.01, 0.2]
	beta	numeric	[0.2, 0.8]
	mu	numeric	[2.0, 8.0]
	epsilon	numeric	[0.01, 1.0]
DBSTREAM	clustering_threshold	numeric	[0.1, 2.0]
	fading_factor	numeric	[0.001, 0.05]
	intersection_factor	numeric	[0.1, 0.8]
	minimum_weight	numeric	[0.5, 5.0]
StreamKMeans	chunk_size	integer	[100, 500]
	n_clusters	integer	[2, 20]
	halflife	numeric	[0.05, 0.8]

VI. RESULTS ANALYSIS

Table III presents a comprehensive comparison of AutoDSC against five baselines across ten diverse real-world and synthetic datasets, evaluating clustering quality with both ARI and CMM. Following OnlineAutoClust's *default-baseline* [16], we summarize the single fixed algorithms by trying all four River algorithms at their default hyperparameters and reporting the best-performing model per dataset (denoted Default(Best)). AutoDSC instead selects and tunes configurations automatically.

AutoDSC achieves the highest overall mean ARI of **0.622**, ahead of every baseline, surpassing the strongest AutoML competitor ConfStream (0.572) by 5.0 points, OnlineAutoClust (0.558) by 6.4 points, RandomSearch (0.534) by 8.8 points, and the Default(Best) baseline (0.517) by 10.5 points, and it attains the best mean rank on both ARI and CMM. AutoDSC ranks first on ARI for 7 of the 10 datasets, with the largest margins on the drifting synthetic streams (e.g. Fast-RBF, Gradual, Expanding all ≥ 0.99, where even the best default reaches only 0.76–0.82) and on the harder real stream PenDigits (0.510 vs. 0.412 for the best default). It is outranked on Shuttle and Vehicle, where a single default algorithm is already near-optimal, and on Merge-Split, where OACE leads narrowly (0.985 vs. 0.963). Figure 3 shows the prequential trajectories, where AutoDSC stays at or above all baselines throughout the stream on both a real (PenDigits) and a synthetic (Expanding) workload, reflecting rapid adaptation as the stream evolves.

Figure 4 isolates AutoDSC's behaviour under drift on two synthetic streams. On *Merge-Split* (a), which contains a discrete merge-then-split event mid-stream (shaded), every adaptive method incurs a temporary drop in ARI and recovers within roughly fifteen windows; AutoDSC sustains the highest stable quality (ARI ≈ 1.0) on either side of the drift and recovers together with the strongest baselines, consistent with the adaptive-exploration recovery analysed in Lemma 2. On *Expanding* (b), where within-cluster variance grows continuously throughout the stream (gradient), AutoDSC reaches near-perfect ARI almost immediately and holds it, whereas ConfStream converges more slowly and OACE and MOStream plateau visibly below it. AutoDSC thus both recovers from abrupt drift and sustains quality under gradual drift through its adaptive exploration alone.

To assess these performance gains statistically, the Critical Difference (CD) diagrams in Figure 5 rank all methods using the Friedman test with the Nemenyi post-hoc procedure ($\alpha = 0.05$, CD$= 2.38$). AutoDSC attains the best (lowest) mean rank on both ARI (1.40) and CMM (2.20), ahead of every baseline. On ARI, AutoDSC is significantly better than RandomSearch, MOStream, and the Default(Best) baseline; its advantage over the strongest AutoML competitors

TABLE III

CLUSTERING QUALITY (ARI AND CMM, ↑ HIGHER IS BETTER) ACROSS DATASETS OVER 10 SEEDS, AS MEAN±STD. BEST PER METRIC PER ROW IN **BOLD**, SECOND BEST UNDERLINED.

Dataset	AutoML						Rule-based		Baselines			
	AutoDSC		ConfStream		OACE		MOStream		RandomSearch		Default(Best)	
	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM	ARI	CMM
Insects	**.117±.003**	.905±.024	.016±.004	.827±.019	.048±.035	**.969±.028**	.090±.045	.964±.006	.054±.007	.880±.019	.100±.002	.947±.001
KDD99	**.037±.000**	.942±.015	.037±.001	.910±.013	.036±.003	.626±.133	.018±.014	.780±.174	.033±.001	.935±.017	.037±.000	**.990±.000**
PenDigits	**.510±.012**	**.888±.014**	.271±.029	.668±.025	.311±.120	.777±.107	.363±.010	.764±.016	.367±.035	.814±.016	.412±.000	.823±.000
Shuttle	.363±.011	.900±.006	.341±.002	.887±.000	.146±.083	**.974±.034**	.002±.001	.380±.054	.270±.014	.903±.018	**.409±.000**	.886±.000
Vehicle	.320±.002	.975±.005	.303±.001	.967±.000	.312±.003	.976±.000	.335±.017	**.990±.001**	.172±.017	.977±.001	**.335±.003**	.986±.000
Expanding	**.993±.001**	**.999±.001**	.953±.028	.969±.020	.895±.298	.900±.300	.938±.111	.970±.058	.844±.017	.927±.011	.756±.103	.868±.055
Gradual	**.998±.001**	1.000±.000	.956±.042	.964±.032	.995±.014	**1.000±.000**	.978±.066	.984±.047	.921±.014	.978±.004	.759±.130	.838±.094
Merge-Split	.963±.004	.988±.001	.952±.009	.979±.007	**.985±.001**	**.999±.000**	.932±.102	.968±.062	.877±.012	.947±.009	.895±.054	.945±.034
Sudden	**.921±.001**	**.956±.001**	.904±.013	.939±.010	.855±.003	.884±.000	.774±.090	.841±.067	.878±.006	.947±.002	.646±.160	.757±.116
Fast-RBF	**.999±.001**	**1.000±.000**	.988±.001	.989±.000	.995±.008	.999±.002	.911±.109	.938±.076	.924±.009	.963±.005	.818±.096	.896±.056
Mean Score	**.622±.374**	**.955±.042**	.572±.391	.910±.094	.558±.399	.910±.116	.534±.392	.858±.179	.534±.367	.927±.048	.517±.287	.894±.071
Mean Rank	**1.40**	**2.20**	3.60	4.30	3.60	2.90	4.10	3.80	4.50	3.50	3.80	4.30

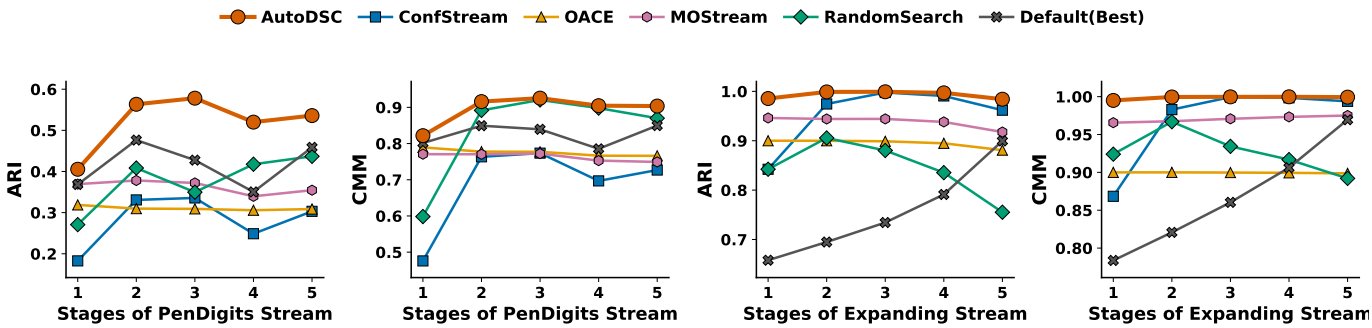


Fig. 3. Prequential clustering quality (ARI and CMM) across stream stages for all methods on a real (PenDigits) and a synthetic drifting (Expanding) stream, with AutoDSC drawn thick.

(ConfStream and OACE, both at rank 3.60) falls just inside the critical difference, so with only ten datasets these are not separated by a statistically significant margin. AutoDSC nonetheless ranks first on both metrics.

Beyond clustering quality, AutoDSC remains practical in throughput and memory. Among the AutoML methods it is the fastest, processing **2.8× more instances/s** than ConfStream and **2.0× more** than OACE, while using less memory than both (≈25 MB vs. 27 and 53 MB). The Default(Best) baseline is naturally faster as it performs no search, and RandomSearch trades quality for speed; AutoDSC instead occupies the favorable region of the throughput–memory plane (Figure 6), pairing the highest clustering quality with a low memory footprint. OACE shows the worst AutoML footprint (53 MB) and MOStream the highest overall (166 MB).

To quantify the contribution of each component, we ran a leave-one-out ablation over all real-world datasets (×10 seeds), disabling one mechanism at a time, namely recency-weighted elite ranking, the adaptive exploration rate, the multi-objective (3-CVI) score, and diversity-preserving pruning. (Replacing the guided PGSSO search by random configuration generation is already covered by the RandomSearch baseline.) Figure 7 reports each variant's score relative to the full model. The multi-objective CVI score is the most impactful component on real-world data, since removing it lowers ARI by ∼2% and CMM by ∼3%, confirming that combining three complementary indices is more robust than any single index. Recency weighting and diversity pruning each give smaller but consistent gains, while the adaptive exploration rate has a negligible standalone effect on these streams. The full configuration is best overall, validating the integrated design.

Finally, AutoDSC adaptively reshapes its population of candidate algorithms as the stream evolves. Figure 8 shows the algorithm composition of the population across stream windows on the Insects dataset, where each cell is the base algorithm occupying a population slot at that window. AutoDSC continuously maintains a diverse mix and shifts the composition between CluStream, DenStream, DBSTREAM and StreamKMeans as the active concept changes, rather than committing to a single algorithm.

A. *Sensitivity Analysis*

We analyze AutoDSC's sensitivity to its two free hyperparameters, window size w and population size K, averaged over all datasets (×10 seeds), jointly reporting clustering quality (ARI) and throughput (Figure 9). Window size shows a clear quality optimum at $w = 1000$, where smaller windows give insufficient data for reliable per-window CVI evaluation, while larger windows delay adaptation to drift and reduce throughput. Population size exhibits a quality plateau, where ARI rises steeply from $K = 5$ and saturates by $K \approx 15$, while throughput falls monotonically as K grows. The knee of this trade-off is at $K = 10$, which attains near-peak quality at substantially higher throughput, justifying it as the default.

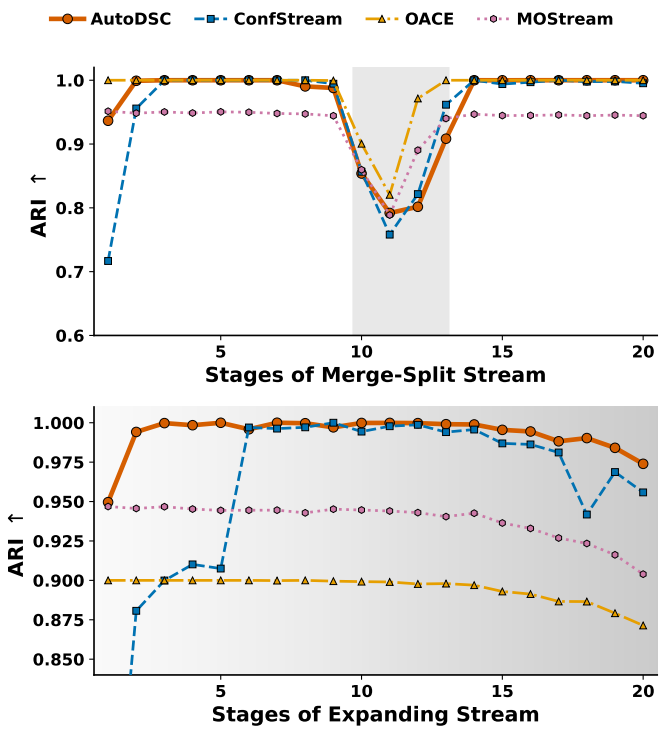
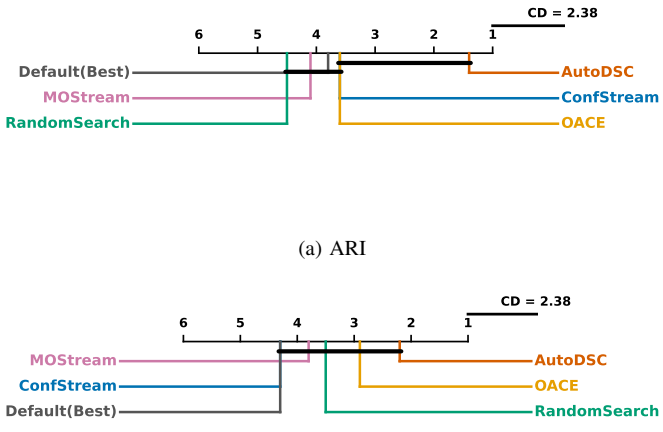


Fig. 4. Per-window ARI (seed-averaged) under drift. (a) *Merge-Split*: a discrete drift event (shaded). (b) *Expanding*: continuous drift (left-to-right gradient).



(a) ARI

(b) CMM

Fig. 5. Critical Difference diagrams (Nemenyi, $\alpha = 0.05$) for ARI and CMM; methods connected by a bar are not significantly different.

Overall, AutoDSC is stable within reasonable ranges and degrades gracefully outside them, with the default configuration ($w = 1000$, $K = 10$) sitting at the quality/efficiency sweet spot of both parameters.

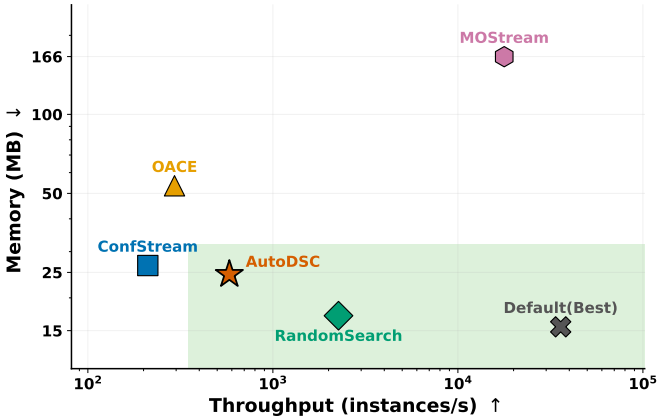


Fig. 6. Throughput vs. memory trade-off (mean over all datasets, log–log axes); the shaded region marks the favorable high-throughput, low-memory area, where AutoDSC (⋆) sits.

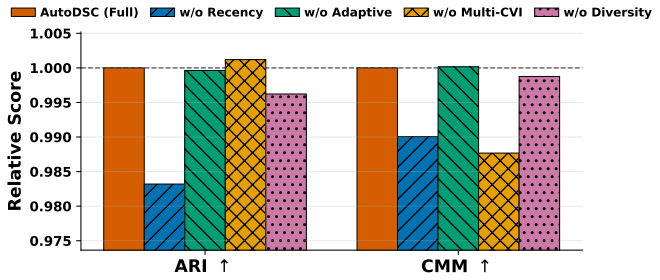


Fig. 7. Leave-one-out ablation on real-world streams, reporting ARI and CMM of each variant relative to the full AutoDSC (dashed line = 1.0).

VII. Discussion

AutoDSC represents a significant advancement in AutoML for streaming environments through several key innovations. Our multi-objective evaluation strategy, combining three normalized clustering validation indices, provides robust performance assessment compared to single-metric approaches used in ConfStream (Silhouette Width only) and OnlineAutoClust's ensemble voting. This prevents optimization convergence to configurations that excel in one aspect while failing catastrophically in others [28], [29]. The PGSSO outperforms existing methods by dynamically adjusting exploration rates based on performance trends, enabling rapid parameter space exploration after concept drift while efficiently exploiting successful configurations during stable periods, contrasting with ConfStream's fixed ensemble strategy and OnlineAutoClust's computationally intensive offline Bayesian optimization [36]. While our current evaluation focuses on tabular data streams with numerical and categorical features, extensions to high-dimensional sparse text streams, multi-modal event streams, or image-based data would require domain-specific clustering algorithms beyond this scope. These novel contributions collectively bridge the gap between traditional AutoML approaches and the dynamic requirements of deploying real-time streaming applications in IoT, sensor networks, and analytics

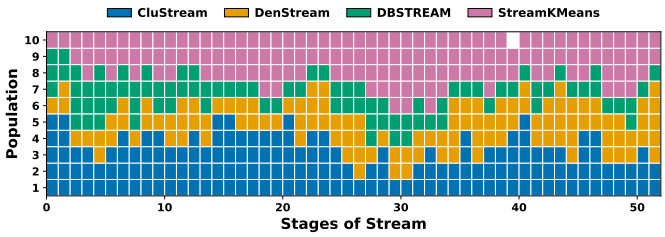


Fig. 8. Evolution of the AutoDSC population composition over the Insects stream, where colors denote the selected base algorithm per population slot (rows) and stream stage (columns).

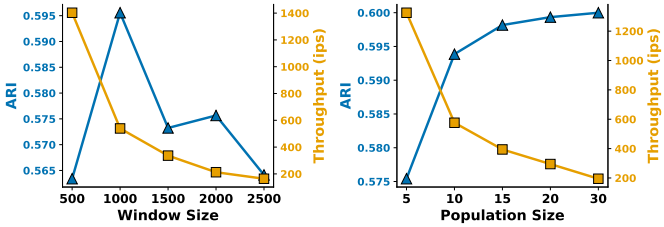


Fig. 9. Sensitivity of AutoDSC to window size (left) and population size (right), reporting ARI (left axis) and throughput (right axis) averaged over all datasets.

platforms, where manual configuration is impractical [18].

VIII. Conclusion and Future Work

We presented AutoDSC, an AutoML framework that solves the combined algorithm selection and hyperparameter optimization problem for data stream clustering in a single pass. AutoDSC proposes configurations with a population-guided search over a recency-weighted elite archive and chooses among them using a label-free multi-objective criterion, adapting implicitly as the stream drifts. Across ten diverse data streams, AutoDSC attains the best mean rank on both ARI and CMM and the highest mean ARI of all methods, a statistically significant improvement over untuned single algorithms, while being the fastest and most memory-efficient of the automated approaches. Future research directions include extending AutoDSC to semi-supervised settings, multi-task clustering, and advanced multi-objective optimization techniques that balance accuracy, efficiency, and robustness in dynamic settings.

References

[1] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.

[2] Y. Poulakis, C. Doulkeridis, and D. Kyriazis, "A survey on automl methods and systems for clustering," *ACM Transactions on Knowledge Discovery from Data*, vol. 18, no. 5, pp. 1–30, 2024.

[3] D. Tschechlov, M. Fritz, and H. Schwarz, "Automl4clust: Efficient automl for clustering analyses," in *Proceedings of the 24th International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org, 2021, pp. 343–348.

[4] Y. Liu, S. Li, and W. Tian, "Autocluster: Meta-learning based ensemble method for automated unsupervised clustering," in *Advances in Knowledge Discovery and Data Mining (PAKDD 2021)*. Springer, 2021, pp. 246–258.

[5] R. ElShawi, H. Lekunze, and S. Sakr, "csmartml: A meta learning-based framework for automated selection and hyperparameter tuning for clustering," in *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 2021, pp. 1119–1126.

[6] D. Treder-Tschechlov, M. Fritz, H. Schwarz, and B. Mitschang, "Ml2dac: Meta-learning to democratize automl for clustering analysis," *Proc. ACM Manag. Data*, vol. 1, no. 2, Jun. 2023. [Online]. Available: https://doi.org/10.1145/3589289

[7] R. ElShawi and S. Sakr, "Tpe-autoclust: A tree-based pipline ensemble framework for automated clustering," in *2022 IEEE International Conference on Data Mining Workshops (ICDMW)*, 2022, pp. 1144–1153.

[8] M. C. da Silva, G. M. Tavares, E. Medvet, and S. B. Junior, "Problem-oriented automl in clustering," *arXiv preprint arXiv:2409.16218*, 2024, arXiv:2409.16218.

[9] L. Yang and A. Shami, "Iot data analytics in dynamic environments: From an automated machine learning perspective," *Engineering Applications of Artificial Intelligence*, vol. 116, p. 105366, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197622003803

[10] A. Bifet, R. Gavaldà, G. Holmes, and B. Pfahringer, *Machine Learning for Data Streams with Practical Examples in MOA*. MIT Press, 2018. [Online]. Available: https://moa.cms.waikato.ac.nz/book/

[11] B. Celik, P. Singh, and J. Vanschoren, "Online automl: An adaptive automl framework for online learning," *Machine Learning*, vol. 112, no. 6, pp. 1897–1921, 2023.

[12] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "Asml: A scalable and efficient automl solution for data streams," in *Proceedings of the Third International Conference on Automated Machine Learning*, ser. Proceedings of Machine Learning Research, K. Eggensperger, R. Garnett, J. Vanschoren, M. Lindauer, and J. R. Gardner, Eds., vol. 256. PMLR, 09–12 Sep 2024, pp. 11/1–26. [Online]. Available: https://proceedings.mlr.press/v256/verma24a.html

[13] Q. Wu, C. Wang, J. Langford, P. Mineiro, and M. Rossi, "Chacha for online automl," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 11 263–11 273. [Online]. Available: https://proceedings.mlr.press/v139/wu21d.html

[14] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "Auto-reg: A dynamic automl framework for streaming regression," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2025, pp. 245–256.

[15] M. Carnein, H. Trautmann, A. Bifet, and B. Pfahringer, "confstream: Automated algorithm selection and configuration of stream clustering algorithms," in *International conference on learning and intelligent optimization*. Springer, 2020, pp. 80–95.

[16] R. El Shawi and D. Rozgonjuk, "Onlineautoclust: A framework for online automated clustering," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 3870–3874.

[17] Z. Wang, X. Wang, and S. Zhang, "MOStream: A modular and self-optimizing data stream clustering algorithm," in *2024 IEEE International Conference on Data Mining (ICDM)*, 2024, pp. 500–509.

[18] M. Bahri, A. Bifet, J. Gama, H. M. Gomes, and S. Maniu, "Data stream analysis: Foundations, major tasks and tools," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 11, no. 3, p. e1405, 2021.

[19] L. Hubert and P. Arabie, "Comparing partitions," *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[20] H. Kremer, P. Kranen, T. Jansen, T. Seidl, A. Bifet, G. Holmes, and B. Pfahringer, "An effective evaluation measure for clustering on evolving data streams," in *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '11. New York, NY, USA: Association for Computing Machinery, 2011, pp. 868–876. [Online]. Available: https://doi.org/10.1145/2020408.2020555

[21] C. C. Aggarwal, "Data streams: models and algorithms," *Springer Science & Business Media*, 2013.

[22] C. C. Aggarwal, S. Y. Philip, J. Han, and J. Wang, "A framework for clustering evolving data streams," in *Proceedings 2003 VLDB conference*. Elsevier, 2003, pp. 81–92.

[23] F. Cao, M. Ester, W. Qian, and A. Zhou, "Density-based clustering over an evolving data stream with noise," in *SIAM International*

Conference on Data Mining (SDM), 2006. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611972740.9

[24] M. Hahsler and M. Bolaños, "Clustering data streams based on shared density between micro-clusters," *IEEE Transactions on Knowledge and Data Engineering*, 2016. [Online]. Available: https://ieeexplore.ieee.org/document/7425260

[25] D. Sculley, "Web-scale k-means clustering," in *WWW 2010, Raleigh, North Carolina, USA*, 2010. [Online]. Available: http://code.google.com/p/sofia-ml

[26] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem *et al.*, "River: machine learning for streaming data in python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.

[27] L. O'Callaghan, N. Mishra, A. Meyerson, S. Guha, and R. Motwani, "Streaming-data algorithms for high-quality clustering," in *Proceedings of the 18th International Conference on Data Engineering (ICDE'02)*, 2002. [Online]. Available: http://www-db.stanford.edu/~loc/icde02.ps

[28] O. Arbelaitz, I. Gurrutxaga, J. Muguerza, J. M. Pérez, and I. Perona, "An extensive comparative study of cluster validity indices," *Pattern recognition*, vol. 46, no. 1, pp. 243–256, 2013.

[29] Y. Liu, Z. Li, H. Xiong, X. Gao, and J. Wu, "Understanding of internal clustering validation measures," in *2010 IEEE International Conference on Data Mining*, 2010, pp. 911–916.

[30] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine learning*, vol. 47, no. 2-3, pp. 235–256, 2002.

[31] N. Cesa-Bianchi and G. Lugosi, *Prediction, learning, and games*. Cambridge university press, 2006.

[32] O. Besbes, Y. Gur, and A. Zeevi, "Stochastic multi-armed-bandit problem with non-stationary rewards," in *Advances in neural information processing systems*, 2014, pp. 199–207.

[33] W. M. Rand, "Objective criteria for the evaluation of clustering methods," *Journal of the American Statistical association*, vol. 66, no. 336, pp. 846–850, 1971.

[34] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine learning research*, vol. 7, no. Jan, pp. 1–30, 2006.

[35] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *The journal of machine learning research*, vol. 13, no. 1, pp. 281–305, 2012.

[36] F. Hinder, A. Artelt, and B. Hammer, "Towards non-parametric drift detection via dynamic adapting window independence drift detection (dawidd)," in *International Conference on Machine Learning*. PMLR, 2020, pp. 4249–4259.